

GRADO EN INGENIERÍA INFORMÁTICA

CURSO 2025-2026



UNIVERSIDAD
DE GRANADA

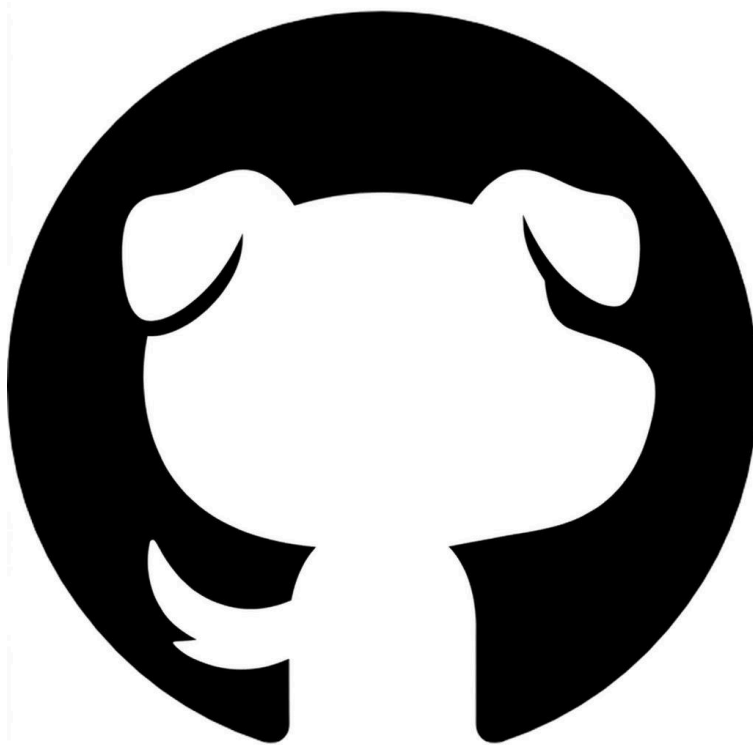
Documentación del proyecto de Tecnologías web. Portal de Gestión de protectora de animales.

Autores Grupo A1, Subgrupo 4:

Pablo Ordóñez Gómez
Ángel Moreno Expósito
Guillermo Moyano García

1. Descripción general - GuauHub.

GuauHub es una plataforma web orientada a una protectora de animales con estética similar a GitHub, enfocada principalmente para informáticos pero apta para todos los públicos. Su objetivo principal es facilitar la adopción responsable de los animales que se encuentran en su protectora asignada, actuando como puente entre los animales del refugio y los potenciales adoptantes, mientras permite al personal y voluntarios llevar un control de los animales, las solicitudes de los posibles adoptantes y actualizar periódicamente el estado de los animales.



2. Objetivos principales

OBJ 1- Centralizar toda la gestión de adopciones en la misma página web.

OBJ 2- Generar una interfaz separada para usuarios normales y voluntarios.

OBJ 3- Gestionar correctamente la puesta en adopción de los animales y su estado en la web.

OBJ 4- Realizar un diseño coherente en toda la web y con un estilo adecuado al tema a cubrir.

OBJ 5- Facilitar la comunicación entre adoptantes y voluntarios.

3. Especificación de requisitos.

Requisitos funcionales:

- **RF-01: Gestión de Sesiones y Autenticación.** El sistema debe permitir el inicio y cierre de sesión de los usuarios. Debe ser capaz de mantener la sesión activa hasta que esta se cierre e identificar al usuario en la esquina superior derecha de la interfaz.
- **RF-02: Control de Roles de Usuario.** El sistema debe diferenciar entre 3 niveles de acceso:
 - *No identificado:* Acceso público al catálogo y fichas
 - *Usuario normal:* Acceso al catálogo, área de adopción y seguimiento
 - *Administrador/Voluntario:* Acceso total y panel de gestión propio
- **RF-03: Visualización del Catálogo Público.** El sistema debe mostrar un listado de animales disponibles, permitiendo acceder a una ficha técnica detallada que incluya fotografías y datos descriptivos.
- **RF-04: Gestión del Inventario de Animales.** El sistema debe permitir a los voluntarios registrar altas, tramitar bajas y modificar la información de los animales, incluyendo actualizaciones sobre su estado de salud.
- **RF-05: Solicitud de Adopción.** El sistema debe proporcionar un formulario formal para que los usuarios registrados soliciten la adopción de un animal específico.
- **RF-06: Área Personal de Seguimiento.** El sistema debe disponer de una sección privada para usuarios registrados donde puedan consultar el estado en tiempo real de sus solicitudes de adopción.
- **RF-07: Panel de Contacto.** El sistema debe disponer de un formulario o sección de contacto general.

Requisitos no funcionales:

- **RNF-01: Diseño Adaptable.** La interfaz gráfica debe ajustarse dinámicamente a las dimensiones de la pantalla del dispositivo del cliente.
- **RNF-02: Accesibilidad Web.** El código generado debe ser accesible para todo tipo de personas sin importar su condición.
- **RNF-03: Arquitectura de la Interfaz.** La estructura visual debe incluir obligatoriamente: menú superior de navegación, zona central para el contenido dinámico, menú lateral contextual, y cabecera/pie de página unificados.
- **RNF-04: Independencia de la Capa de Presentación.** Todos los estilos visuales deben estar definidos en archivos CSS externos.
- **RNF-05: Pie de página.** El pie de página global debe incluir, de manera persistente, enlaces dirigidos al formulario de contacto (con datos de los desarrolladores) y al documento técnico del proyecto.
- **RNF-06: Validación Nativa de Formularios.** Los datos introducidos en los formularios deben validarse para asegurar que sean correctos. Todos los campos de los formularios deben ser obligatorios.

4. Diseño de la base de datos.

La base de datos corresponde a los cimientos de la página, gracias a ella podemos almacenar los datos de los usuarios, los animales y las solicitudes, entre otros datos importantes. Ha sido implementada con la herramienta mysql mediante las migraciones de laravel.

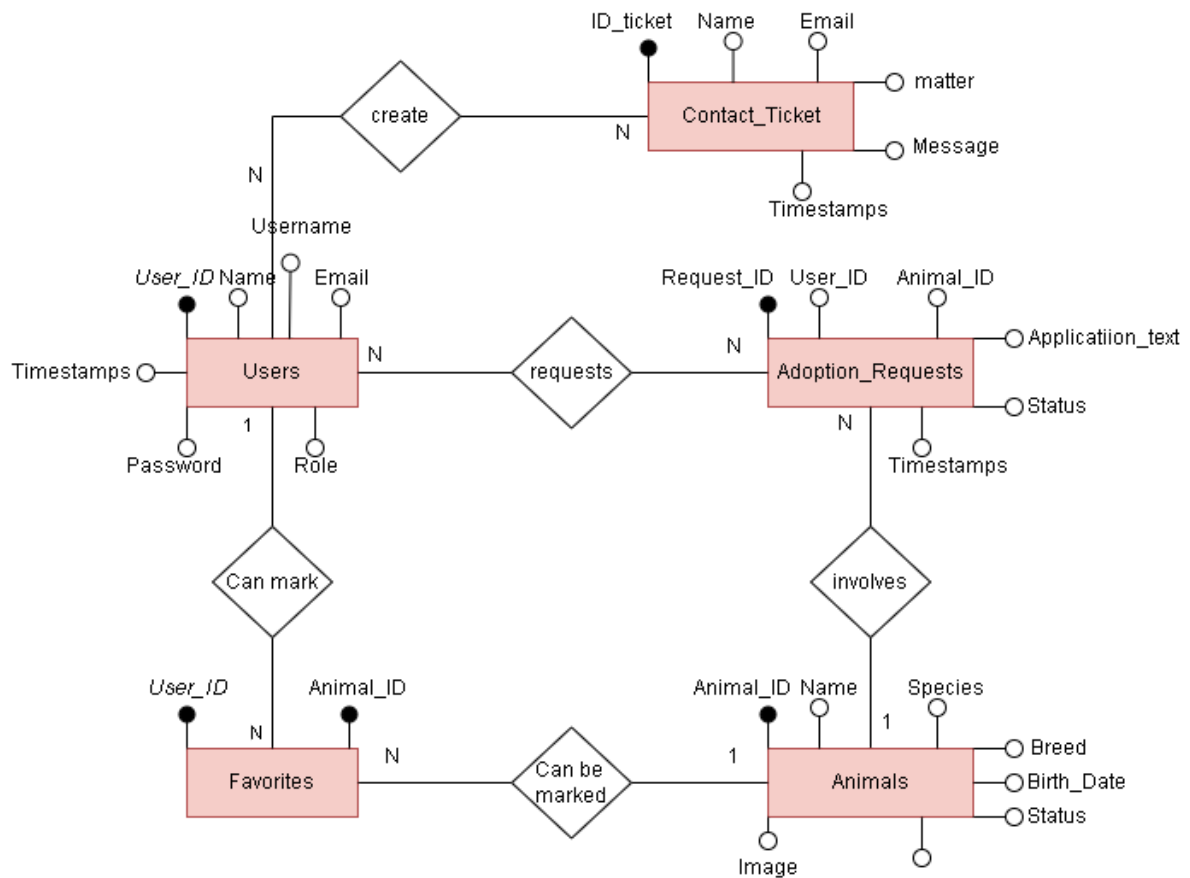
4.1 Tablas necesarias.

Para el correcto funcionamiento de nuestra página web hemos considerado las siguientes tablas:

- **Usuarios:** Guardará la información de un usuario al registrarse generando su id correspondiente. Sus campos son: **Nombre, nombre de usuario, email, contraseña y rol, que podrá ser “usuario” o “voluntario”.**
- **Animales:** Guardaremos los datos importantes de cada animal al introducirlo en nuestro sistema para que el usuario pueda disponer de la información mínima necesaria sobre él. Sus campos son: **ID, Nombre, especie, sexo, fecha de nacimiento, estado de salud y la imagen asociada al animal.**
- **Solicitud de adopción:** Guardaremos las solicitudes de adopción entre un determinado usuario y un animal. Sus campos son: **ID_solicitud, ID_usuario, ID_animal, Texto de la solicitud y estado de la solicitud(“pendiente, aprobada o rechazada”).**
- **Favoritos:** Guardaremos una lista de selecciones de animales por un usuario como favoritos. Sus campos son: **ID_Usuario, ID_Animal.**
- **Solicitud de contacto:** Guardaremos en la base de datos todas las solicitudes de los formularios de contacto que nos lleguen, para contestar al formulario no hace falta estar registrado así que no guardaremos el ID del usuario que los rellena. Sus campos son: **ID_solicitud, nombre,email, asunto y mensaje**

Todos estos campos a la hora de la implementación se gestionarán en inglés por la preferencia de laravel por este idioma a la hora de realizar las migraciones.

4.2 Diseño relacional completo de la base de datos.



4.3 Implementación de la base de datos en laravel.

En laravel, las bases de datos se implementan siguiendo el modelo vista-controlador.

Para esta documentación utilizaremos como ejemplo la tabla animal, el resto se implementan todas de la misma forma.

Por lo tanto distinguimos 3 tipos de archivos:

- **migraciones:** Son básicamente la forma de escribir las tablas usando php, aquí definimos los distintos campos, sus tipos y les damos sus correspondientes características como si se trata de una clave primaria, clave externa, clave compuesta entre otros.

```
public function up(): void
{
    Schema::create('animals', function (Blueprint $table) {
        $table->id(); // id del animal
        $table->string('name'); // nombre del animal
        $table->string('species'); // especie del animal
        $table->string('breed')->nullable(); // raza del animal (puede ser null al no se
        $table->date('birth_date'); // fecha de nacimiento
        $table->enum('sex', ['male', 'female']); // sexo del animal
        $table->text('health_status'); // estado de salud del animal, vacunas etc.
        $table->text('description'); // descripción general corta del animal
        $table->string('image_path'); // Ruta de la imagen del animal
        $table->enum('status', ['available', 'adopted', 'in_process'])->default('availa
        $table->timestamps(); ➔ Tab to Jump
    });
}

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('animals');
}
```

La función up se ejecutará tras ejecutar **php artisan migrate** y se encarga de crear las tablas traduciendo el código introducido en sentencias SQL que puedan ser leídas por la base de datos.

También se crea una función down para eliminar la tabla.

- **Modelos:** Los modelos es lo que nos permite comunicarnos con la tabla de datos desde php sin necesidad de escribir sentencias SQL directamente a la base de datos ya que laravel las traduce automáticamente. Son utilizados en todo momento por los controladores para gestionar las acciones de la página sobre la base de datos.

```

class Animal extends Model
{
    use HasFactory;

    protected $fillable = [
        'name',
        'species',
        'breed',
        'birth_date',
        'sex',
        'health_status',
        'description',
        'image_path',
        'status',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'birth_date' => 'date',
        ];
    }
}

```

- **Controladores:** Se encargan de gestionar la lógica de todo lo que ocurre en la base de datos. Basándose en el modelo y en la migración de la base de datos, se encargan de aplicar las acciones sobre ella de forma compacta y segura. Básicamente se ocupan de modificar los datos de la base de datos para que esta tome forma y funcione como queremos, según las distintas acciones que vamos haciendo en la página.

```

public function store(Request $request)
{
    if (auth()->user()->role !== 'voluntario') {
        return redirect('/catalogo')->withErrors(['error' => 'No tienes permiso para cre
    ]);

    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'species' => 'required|string|max:255',
        'breed' => 'nullable|string|max:255',
        'birth_date' => 'required|date|before_or_equal:today',
        'sex' => 'required|in:male,female',
        'health_status' => 'required|string',
        'description' => 'required|string',
        'image_path' => 'required|url',
    ]);

    $validated['status'] = 'available';

    $animal = Animal::create($validated);

    return redirect('/animal/' . $animal->id)->with('success', 'Repositorio (Animal) cre
}

```

En este ejemplo ilustramos la función del controlador asociada a la acción de crear y guardar un nuevo animal en la base de datos. El controlador se encarga de que el usuario tenga el rol de voluntario, así como de la obligatoriedad de los campos, el tipo de estos y gestiones de seguridad como por ejemplo, impedir que se introduzcan fechas futuras como fecha de nacimiento de un animal.

Una vez se ha hecho todo, da la señal de crear un Animal con esas características y redirecciona al usuario a la página del animal con un mensaje de éxito.

5. Diseño de la página

5.1. Visión General de la Funcionalidad y Roles

A nivel técnico, el frontend está construido prescindiendo de frameworks CSS externos (como Bootstrap) y sin depender de JavaScript para la interactividad. Toda la interfaz, la responsividad y los modales emergentes funcionan exclusivamente mediante HTML5 semántico, directivas de Blade y Vanilla CSS avanzado.

La aplicación implementa un sistema de control de acceso estricto basado en roles, lo que define la visibilidad y capacidad de interacción en cada sección:

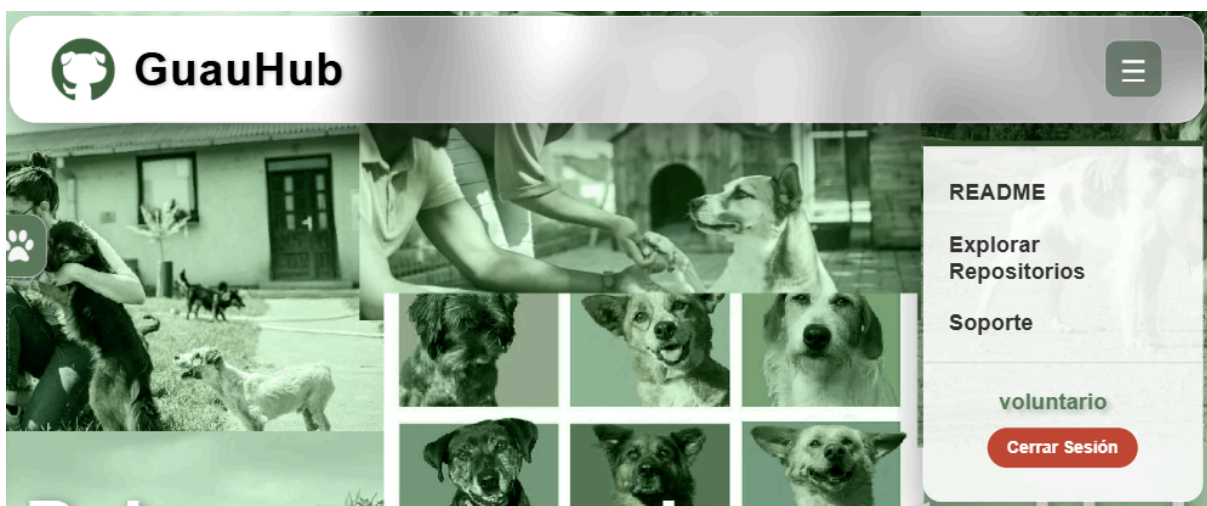
- **Usuarios No Registrados (Invitados):** Tienen acceso público a la carta de presentación de la protectora. Pueden visualizar la página de inicio, explorar el catálogo público de animales y contactar con el equipo a través del formulario de soporte. No pueden iniciar procesos de adopción ni guardar favoritos.
- **Adoptantes (Usuarios estándar registrados):** Tienen acceso completo al catálogo interactivo. Se les desbloquea la capacidad de guardar animales en favoritos y enviar formularios de adopción. Además, tienen acceso a su panel personal para gestionar sus solicitudes pendientes y modificar la información de su perfil.
- **Voluntarios (Administradores/Staff):** Poseen el control total de la base de datos y flujos de adopción. Pueden crear, editar y eliminar animales del catálogo. Tienen acceso exclusivo a las bandejas de entrada para aceptar o rechazar adopciones y gestionar los tickets de soporte enviados por el público.

5.2. Componentes Globales y Páginas Generales

5.2.1. Cabecera de Navegación (Header)

- **Funcionalidad General:** Es el componente fijo superior que permite la navegación principal por la aplicación y la autenticación. Visualmente, está estructurado de izquierda a derecha: en el extremo izquierdo se encuentra el logotipo de GuauHub junto al nombre de la marca; a la derecha se alinea el menú de navegación principal, compuesto por tres enlaces fijos que llevan a "README" (Inicio), "Explorar Repositorios" (Catálogo) y "Soporte" (Contacto); finalmente, en el extremo derecho, se sitúa la zona de usuario con los botones de "Iniciar Sesión" y "Registrarse", o el nombre de perfil si ya se ha autenticado.

- **Visibilidad por Rol:**
 - *Invitados:* Ven los enlaces públicos y los botones de "Iniciar Sesión" y "Registrarse".
 - *Adoptantes/Voluntarios:* Los botones de inicio de sesión se ocultan. En su lugar, se inyecta dinámicamente el nombre del usuario y el botón de "Cerrar Sesión". Si es voluntario, el nombre de usuario se muestra en verde corporativo para distinguirlo.
- **Naturaleza Dinámica:** Utiliza las directivas `@auth` y `@guest` de Blade para renderizar condicionalmente los botones de sesión frente al nombre del usuario. Utiliza la inyección `Auth::user()->username` y una condicional ternaria en la clase CSS para cambiar el color del nombre según el rol.
- **Uso de Modales: Sí.** Contiene dos modales (Login y Registro) controlados por checkboxes ocultos (`#toggle-login`, `#toggle-registro`). El modal de Login tiene estilo verde cristalino, mientras que el de Registro es oscuro. Se renderizan ocultos (`opacity: 0; visibility: hidden`) y aparecen al pulsar sus etiquetas correspondientes.
- **Diseño Adaptativo (Responsive Design):**
 - * *Escritorio (>990px):* Se presenta como una barra horizontal con alineación *space-between*. El menú de navegación se expande linealmente.
 - *Móvil y Tablet (<990px):* La navegación horizontal se oculta. Aparece un "Botón Hamburguesa" (controlado también por un *checkbox hack* `#toggle-menu`). Al pulsarlo, el menú de navegación (`nav`) cambia su dirección a `flex-direction: column` y se despliega verticalmente debajo del logo con una animación tipo persiana (`clip-path: inset()`).



5.2.2. Menú Lateral (Sidebar / Dashboard)

- **Funcionalidad General:** Actúa como el panel de control (Dashboard) personalizado del usuario, situado siempre en el lateral izquierdo de la

pantalla. Es el punto de acceso rápido para la gestión de entidades. Dependiendo del rol, muestra una cabecera de bienvenida y una lista de enlaces verticales con iconos representativos que dirigen a páginas de gestión interna (como la lista de favoritos, el panel de solicitudes pendientes, o el formulario para añadir un nuevo animal).

- **Visibilidad por Rol:**

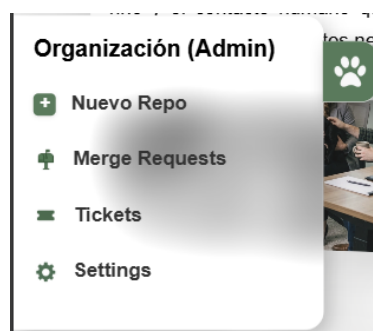
- *Invitados:* Solo ven un bloque de texto estático invitándoles a iniciar sesión para acceder a las funcionalidades de Favoritos y Pull Requests.
- *Adoptantes:* Ven una lista de enlaces para "Repositorios Starred", "Mis Pull Requests" y "Settings".
- *Voluntarios:* Se les ocultan las opciones de adoptante y se muestra un panel exclusivo con enlaces a "Nuevo Repo", "Merge Requests", "Tickets" y "Settings".

- **Naturaleza Dinámica:** 100% dinámico. Toda la estructura está condicionada por múltiples directivas de Blade (@guest, @auth, @if(Auth::user()->role === 'X')) que evalúan el rol del usuario en la sesión para inyectar solo el HTML correspondiente a sus permisos.

- **Uso de Modales:** No utiliza ventanas modales, pero sí actúa como un elemento emergente en su versión móvil.

- **Diseño Adaptativo (Responsive Design):**

- *Escritorio (>990px):* Se posiciona a la izquierda, compartiendo espacio con el contenedor principal usando *Flexbox*. Tiene bordes redondeados a la derecha y una sombra de elevación.
- *Móvil y Tablet (<990px):* Se extrae del flujo del documento mediante *position: fixed* y se esconde a la izquierda de la pantalla usando un margen negativo (*margin-left: -250px*). Se inyecta un botón flotante (pestaña con icono de huella) que sobresale a la derecha. Al pulsar la huella (usando un *checkbox*), el margen pasa a 0 y la barra se desliza sobre la pantalla cubriendo parte del contenido.



5.2.3. Pie de Página (Footer)

- **Funcionalidad General:** Cierra la estructura visual en la parte inferior de todas las páginas, ofreciendo navegación secundaria y contacto. Se divide en tres columnas informativas: la primera contiene el logotipo y el eslogan corporativo; la segunda, la documentación con el pdf “como_se_hizo” y un enlace especial a "Conócenos" para presentar al equipo fundador; y la tercera, la información de contacto de la protectora.
- **Visibilidad por Rol:** Estático. Visible y exactamente igual para todos los usuarios, independientemente de su estado de autenticación o permisos.
- **Naturaleza Dinámica:** Componente estático renderizado mediante Blade (@include('layouts.footer')). Sus enlaces apuntan siempre a rutas estáticas predefinidas en Laravel.
- **Uso de Modales:** No utiliza ventanas modales.
- **Diseño Adaptativo (Responsive Design):**
 - *Escritorio (>990px):* Se divide en tres columnas alineadas horizontalmente usando display: flex con justificación equilibrada (space-between). Mantiene un efecto *glassmorphism* similar al Header.
 - *Móvil (<990px):* Aprovechando flex-wrap: wrap, las tres columnas colapsan en una sola columna vertical (flex-direction: column implícito por falta de espacio). Se eliminan los bordes y sombras para integrarse mejor en pantallas reducidas.

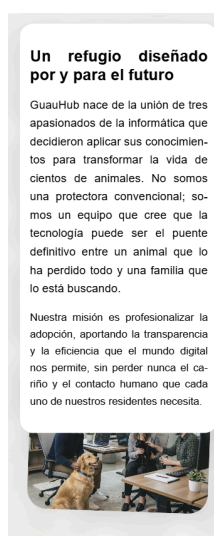


5.2.4. Página Principal (Inicio / Welcome)

- **Funcionalidad General:** Es la primera pantalla que ve el usuario al entrar en el dominio raíz (/). Está diseñada para atrapar la atención, explicar el original modelo de adopción inspirado en "repositorios" de GitHub y dirigir al usuario hacia el catálogo o el registro. Está estructurada en tres grandes bloques visuales: un bloque superior con una imagen gigante de fondo y el eslogan

principal, seguido de bloques de contenido en zigzag que explican por qué adoptar aquí, y finalmente un gran botón para comenzar a explorar los animales de la protectora.

- **Visibilidad por Rol:** Visible para todos los usuarios, aunque su objetivo principal son los invitados.
- **Naturaleza Dinámica:** Principalmente estática. Su función es puramente visual y de marketing, sin inyectar datos de la base de datos.
- **Uso de Modales:** No utiliza modales propios, pero incluye botones que redirigen al catálogo o activan indirectamente los modales de sesión del Header.
- **Diseño Adaptativo (Responsive Design):**
 - *Escritorio (>900px):* Utiliza un diseño de solapamiento ("Overlap") en zigzag. Una caja de texto oscura y ancha se solapa parcialmente (z-index: 10) sobre una imagen gigante inclinada (transform: rotate()). Esta estructura se alterna (texto a la izquierda/imagen a la derecha y viceversa) para guiar la lectura. Los contenedores de estadísticas se alinean horizontalmente.
 - *Móvil (<900px / <480px):* El efecto de solapamiento horizontal se deshace. Mediante flexbox (flex-direction: column-reverse), las imágenes de los perros pasan a situarse en la parte superior y el texto explicativo se coloca justo debajo cubriendo el 100% del ancho de la pantalla. El tamaño de la fuente (font-size) de los títulos principales baja dinámicamente de 2.5rem a 1.5rem y se fuerza la ruptura de palabras largas (overflow-wrap: break-word) para evitar desbordamientos horizontales.



5.3. Páginas Exclusivas de Voluntario (Administración)

5.3.1. Panel de Solicitudes Pendientes (Merge Requests)

- **Funcionalidad General:** Es la bandeja de entrada donde los voluntarios revisan todas las peticiones de adopción enviadas por los usuarios. Visualmente, presenta una cabecera de título y una lista de tarjetas apiladas. Cada tarjeta muestra a la izquierda la foto en miniatura del animal solicitado; en el centro, el nombre del adoptante, el animal que desea y la "justificación" o motivo por el que quiere adoptar; y a la derecha, dos grandes botones de acción: uno verde para "Aceptar" la solicitud (que asigna el animal a ese usuario y lo quita de adopción) y uno rojo para "Rechazar" (que descarta la petición).
- **Visibilidad por Rol:**
 - *Invitados y Adoptantes:* Bloqueado (Redirigidos si intentan acceder).
 - *Voluntarios:* Tienen acceso total a esta vista a través del enlace "Merge Requests" en su Sidebar.
- **Naturaleza Dinámica:** Altamente dinámica. Utiliza un bucle `@foreach` de Blade para iterar sobre la variable `$solicitudes` (que contiene las adopciones pendientes de la base de datos). Por cada iteración, se genera una tarjeta que muestra la foto del animal, el nombre del adoptante y el texto de la solicitud. Si la lista está vacía, se inyecta un mensaje de "No hay solicitudes pendientes".
- **Uso de Modales: Sí.** Cada botón de acción rápida activa una ventana modal estricta de confirmación superpuesta en la pantalla, asegurando que el voluntario no apruebe o rechace una solicitud por accidente.
- **Diseño Adaptativo (Responsive Design):**
 - *Escritorio:* Las solicitudes se listan en formato de tarjetas anchas, con la imagen del animal a la izquierda, la información en el centro y los botones de acción alineados a la derecha.
 - *Móvil:* La tarjeta colapsa en una estructura de columna. La imagen y el texto se apilan, y la botonera de acciones (`.form-actions`) ocupa el 100% del ancho en la parte inferior, con los botones centrados para facilitar ser pulsados con el dedo pulgar.



5.3.2. Gestión de Repositorios (Crear y Editar Animal)

- **Funcionalidad General:** Son dos vistas gemelas que muestran un formulario completo estructurado en un bloque central. Ambos formularios contienen campos de texto y selectores desplegable para introducir los datos vitales del animal: Nombre, Edad, Tipo (Perro/Gato/Otro), Descripción detallada y un campo para subir su Fotografía. En la parte inferior, el formulario de Creación tiene un botón verde para "Guardar", que inserta el nuevo animal en el catálogo público. Por su parte, el formulario de Edición cuenta con el botón de Save Changes y, además, un botón rojo secundario diseñado exclusivamente para eliminar al animal de la base de datos por completo.
- **Visibilidad por Rol:**
 - *Invitados y Adoptantes:* Acceso denegado.
 - *Voluntarios:* Acceden desde el botón "+ Nuevo Repo" del Sidebar (para crear) o desde el botón "Settings/Edit" dentro de la ficha de un animal concreto (para editar).
- **Naturaleza Dinámica:** Dinámica transaccional. En la vista de Creación se usa el token @csrf para seguridad en el envío. En la vista de Edición, los campos del formulario se rellenan automáticamente inyectando los datos de la base de datos (ej: value="{{ \$animal->nombre }}").
- **Uso de Modales: Sí, Crítico.** En la vista de "Editar", existe un botón para "Eliminar Repositorio". Al pulsarlo, no borra directamente, sino que despliega un modal de confirmación final para evitar pérdidas de datos accidentales.
- **Diseño Adaptativo (Responsive Design):**
 - *Escritorio:* El formulario se presenta en una caja centrada (max-width: 600px) con los campos dispuestos verticalmente y márgenes amplios para una lectura cómoda.
 - *Móvil:* La caja pierde sus márgenes laterales y se adapta al 100% del ancho de la pantalla, integrándose completamente en los bordes para maximizar el espacio de los inputs de texto.

The screenshot shows a web form titled "Issues (Estado de Salud / Vacunas)". It contains several sections: a text area for "Vacunación al día. Microchip instalado. Castrado. Leve alergia al pollo.", a "README (Descripción)" section with a text area containing a playful description of a Beagle, and an "Image URL (Ruta de la Imagen)" section with a text input field containing a URL from Unsplash. Below the URL field is a small image of a dog. At the bottom of the form are three buttons: a green "Save changes" button, a black "Cancel" button, and a red "Delete this repository" button.

5.3.3. Bandeja de Soporte (Tickets de Contacto)

- **Funcionalidad General:** Es un panel de administración estructurado visualmente en forma de tabla HTML. Cada fila de la tabla representa un mensaje enviado por un usuario desde la página de Soporte. Las columnas muestran de izquierda a derecha: el nombre del remitente, su correo electrónico, el asunto del mensaje, el cuerpo del texto completo y el estado actual del ticket.
- **Visibilidad por Rol:** Exclusiva para Voluntarios, accesible desde el Sidebar ("Tickets").
- **Naturaleza Dinámica:** Dinámica. Se itera mediante Blade sobre los registros de contacto de la base de datos para generar las filas de la tabla de forma automática.
- **Uso de Modales:** No utiliza modales.
- **Diseño Adaptativo (Responsive Design):**
 - *Escritorio:* Se muestra como una tabla HTML tradicional (<table>, <tr>, <th>, <td>), con cabeceras marcadas en color gris y bordes limpios para visualizar múltiples tickets de un vistazo.
 - *Móvil (<768px):* Las tablas HTML se rompen en móvil. Por ello, se aplica una técnica CSS donde los <tr> se transforman en bloques tipo tarjeta (display: block). Cada celda (<td>) se apila verticalmente, e inyecta dinámicamente el nombre de su columna original a la izquierda usando el pseudo-elemento ::before y el atributo content: attr(data-label), transformando la tabla en una lista de tarjetas de lectura vertical.

NOMBRE
ANGEL
EMAIL
angelmorepo@gmail.com
ASUNTO
Dudas Generales
MENSAJE
Hola queria preguntarle muchas cosas pero ahora no se me ocurre ninguna
FECHA
13/05/2026 19:05

5.4. Páginas de Adoptante e Interacción Principal

5.4.1. Catálogo Principal (Explorar Repositorios)

- **Funcionalidad General:** Es el núcleo de la aplicación, el lugar donde se listan todos los animales en busca de hogar. Visualmente, se presenta como una gran cuadrícula de tarjetas estilo "repositorio". Cada tarjeta muestra una foto recortada en la parte superior, y debajo, información rápida: nombre, una etiqueta indicando si es "Perro" o "Gato" u "Otro", su edad y estado. Si el

usuario está registrado, en la esquina superior de la foto aparece un pequeño botón interactivo con forma de "Estrella" para guardar al animal. Al pulsar sobre "ver detalles de la rama", el usuario es redirigido a la vista de detalle completa de ese animal.

- **Visibilidad por Rol:**

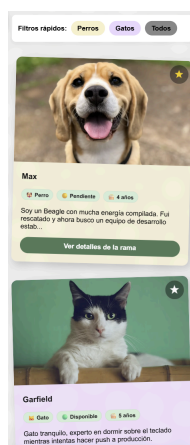
- *Invitados/Voluntarios:* Pueden ver todas las tarjetas y entrar en los detalles, pero el botón de la "Estrella" (Favoritos) no les aparece.
- *Adoptantes:* Ven todas las tarjetas junto con el botón de la "Estrella" para guardarlos en su lista de seguimiento.

- **Naturaleza Dinámica:** Generada dinámicamente. Utiliza @foreach sobre la base de datos de animales. Si un animal ya ha sido adoptado, se filtra para no aparecer en esta lista (o se le añade una etiqueta visual). Además, evalúa si el animal ya está en los favoritos del usuario para rellenar la estrella de amarillo o dejarla en gris vacío.

- **Uso de Modales:** No utiliza modales en esta vista.

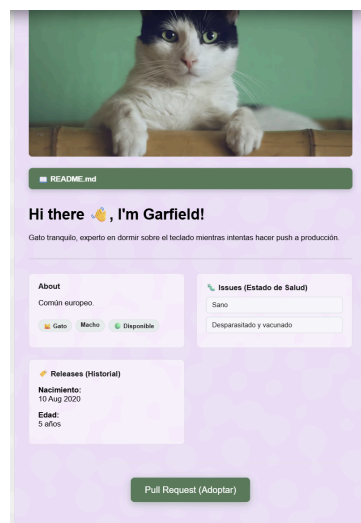
- **Diseño Adaptativo (Responsive Design):**

- *Escritorio:* Las tarjetas se distribuyen usando CSS Grid (grid-template-columns: repeat(auto-fit, minmax(280px, 1fr))), generando entre 3 y 4 columnas dependiendo del tamaño del monitor.
- *Móvil/Tablet:* Gracias al auto-fit de CSS Grid, al reducir el tamaño de la pantalla, las columnas colapsan fluidamente hasta quedar en 2 columnas (Tablet) o en 1 sola columna vertical (Móviles pequeños), expandiendo la tarjeta al ancho completo de la pantalla para una fácil lectura.



5.4.2. Vista Detalle de Repositorio (Perfil del Animal)

- **Funcionalidad General:** Es la ficha individual ampliada de un animal concreto. Visualmente, se divide en dos bloques: En la zona superior, se ve la imagen del animal junto con su “readme”, una descripción del mismo. En la zona inferior se ven los datos del animal como su raza, su fecha de nacimiento, sus datos sanitarios, y abajo del todo un gran botón verde para adoptar si eres usuario normal, o editar si eres voluntario.
- **Visibilidad por Rol:**
 - *Invitados:* Ven la información y la foto, pero los botones "Create Pull Request" y "Star" son sustituidos por un mensaje que les insta a iniciar sesión.
 - *Adoptantes:* Ven y pueden usar los botones de adopción y favoritos.
 - *Voluntarios:* Además de ver la ficha, se les inyecta un botón extra "Edit" en la parte inferior que les redirige directamente a la vista de edición de ese animal.
- **Naturaleza Dinámica:** Inyecta todos los datos del modelo Animal usando Blade (`{{ $animal->descripcion }}`, etc).
- **Uso de Modales: No.**
- **Diseño Adaptativo (Responsive Design):**
 - *Escritorio:* Presentación en Flexbox (lado a lado), imagen a la izquierda y texto a la derecha.
 - *Móvil:* Colapso a una columna vertical (flex-direction: column). La gran imagen pasa a ocupar el ancho superior completo y el bloque de texto baja, quedando el botón de "Pull Request" posicionado como CTA final al terminar de hacer scroll.



5.4.3. Panel de "Starred Repos" (Favoritos) y "Mis Pull Requests"

- **Funcionalidad General:** Son dos vistas accesibles desde el Dashboard (Sidebar) del adoptante. "Starred Repos" muestra una cuadrícula idéntica a la del catálogo, pero filtrada exclusivamente con los animales guardados por el usuario. "Mis Pull Requests" muestra una lista o tabla con los animales que el usuario ha solicitado adoptar, indicando visualmente el estado de su petición (Pendiente en amarillo) además deja editar esta.
- **Visibilidad por Rol:**
 - *Exclusivo para Adoptantes.* Los invitados no pueden acceder, y los voluntarios tienen su propio panel de gestión (Merge Requests).
- **Naturaleza Dinámica:** Completamente vinculada a las tablas pivote del usuario en la base de datos (relación muchos a muchos entre Usuario y Animal).
- **Uso de Modales:** No suelen utilizar modales. Si el usuario desea retirar un Favorito, pulsa la estrella directamente y la recarga de página lo elimina de la lista.
- **Diseño Adaptativo (Responsive Design):** Heredan el diseño responsive del catálogo (Grid auto-ajutable) para favoritos, y el diseño de tarjetas apilables verticalmente para las solicitudes.

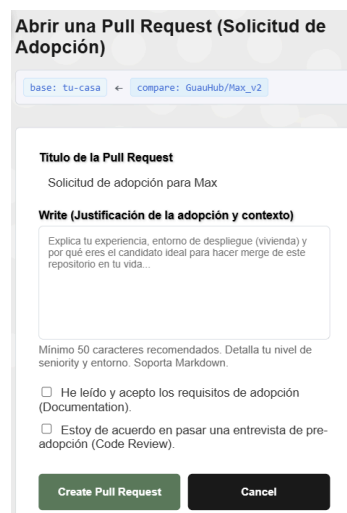


5.4.4. Formulario de Adopción (Open Pull Request)

- **Funcionalidad General:** Es la página a la que se llega tras pulsar "Create Pull Request" en el perfil de un animal. El usuario (adoptante) debe escribir en un area la justificación de por qué es el candidato ideal. Finalmente, en la

esquina inferior derecha, hay un gran botón verde "Create pull request" para enviar el formulario.

- **Visibilidad por Rol:** Exclusivo para Adoptantes registrados. Los invitados son redirigidos al login.
- **Naturaleza Dinámica:** Inyecta el nombre del animal en las etiquetas de las ramas mediante Blade (`{{ $animal->name }}`). Al enviar el formulario (POST), guarda la solicitud en la base de datos bajo la tabla `adoption_requests`.
- **Uso de Modales:** No utiliza modales; es una vista completa de formulario.
- **Diseño Adaptativo (Responsive Design):**
 - *Escritorio:* El formulario se presenta centrado con márgenes grises alrededor, simulando la interfaz de GitHub.
 - *Móvil:* Los márgenes desaparecen para aprovechar el 100% de la pantalla (`width: 100vw`). El cuadro comparativo de ramas adapta la flecha a una disposición apilada si no hay espacio horizontal suficiente.



5.4.5. Settings / Mi Perfil (Adoptante)

- **Funcionalidad General:** La página de gestión de la cuenta personal. Visualmente, presenta un gran contenedor central. En la parte superior aparece un avatar circular del usuario. Debajo, un formulario estructurado en filas con campos de texto para actualizar los datos personales: "Username", "Email", "Nombre" (nombre que le saldrán a los voluntarios) además de la contraseña. Termina con un botón verde en la zona inferior para "Guardar cambios" y otro para "Actualizar Contraseña".
- **Visibilidad por Rol:** Cualquier usuario autenticado (tanto Adoptantes como Voluntarios tienen su propio perfil).

- **Naturaleza Dinámica:** Los inputs (campos de texto) se autorellenan recuperando los datos del usuario logueado usando `Auth::user()->...` como valor inicial (ej: `value="{{ Auth::user()->email }}"`). El formulario envía un POST al controlador para actualizar la BBDD.
- **Uso de Modales:** No.
- **Diseño Adaptativo (Responsive Design):**
 - *Escritorio:* Bloque centralizado con márgenes amplios (max-width).
 - *Móvil:* Eliminación de los márgenes laterales (width: 100vw) para que los campos de texto del formulario (inputs) sean lo más grandes posible en las pantallas táctiles.

La imagen muestra una interfaz de usuario con dos secciones principales. La primera sección, titulada 'AJUSTES PÚBLICOS' y 'Perfil de Usuario', contiene campos para 'Nombre Público' (con el valor 'adoptante'), 'Nombre de Usuario' (con el valor 'adoptante') y 'Correo Electrónico' (con el valor 'adoptante@gmail.com'). Hay un botón verde 'Guardar Cambios'. La segunda sección, titulada 'SEGURIDAD' y 'Cambiar Contraseña', contiene campos para 'Contraseña Actual', 'Nueva Contraseña' y 'Confirmar Nueva Contraseña', con un botón verde 'Actualizar Contraseña'.

5.5. Páginas Estáticas y de Soporte

5.5.1. Soporte (Contacto)

- **Funcionalidad General:** Un formulario de contacto público para dudas generales. Visualmente, está envuelto en una tarjeta blanca o contenedor central. Pide el Nombre, Correo electrónico, Asunto (Subject) y un Mensaje (textarea). Al final, un botón verde de "Send Message" para registrar el ticket en el sistema.
- **Visibilidad por Rol:** Accesible para todos (Invitados, Adoptantes, Voluntarios).
- **Naturaleza Dinámica:** Formulario de envío estándar. Si el usuario está autenticado, los campos de Nombre y Correo pueden autocompletarse con `Auth::user()` para agilizar el envío.
- **Uso de Modales:** No utiliza modales.
- **Diseño Adaptativo (Responsive Design):** Funciona igual que los formularios de Creación de Animales: caja centrada en escritorio, ancho completo del dispositivo en móvil.

Soporte (Contacto)

¿Tienes dudas sobre cómo hacer un fork a alguno de nuestros animales o sobre el proceso de adopción (Pull Request)? ¡Abre un ticket de soporte enviándonos este formulario!

Nombre del desarrollador (Tú)

Correo electrónico

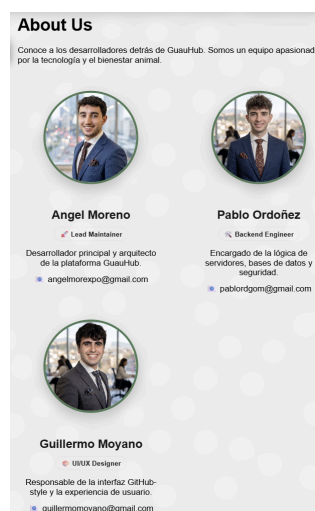
Asunto del Issue

Descripción detallada

[Enviar Ticket de Soporte](#)

5.5.2. Conócenos (About Us / Equipo)

- **Funcionalidad General:** Página estática corporativa que presenta la misión de la protectora y al equipo desarrollador. Visualmente, suele presentar un gran bloque de texto inicial (misión y valores) seguido de una cuadrícula de fotos circulares de perfil con los nombres y redes sociales (GitHub/LinkedIn) de cada miembro del equipo.
- **Visibilidad por Rol:** Pública (Todos los roles).
- **Naturaleza Dinámica:** 100% estática. No consume datos de la base de datos, toda la información está directamente en el archivo HTML/Blade.
- **Uso de Modales:** No utiliza modales.
- **Diseño Adaptativo (Responsive Design):**
 - *Escritorio:* Las fotos del equipo se muestran alineadas en fila (usando Flexbox o Grid).
 - *Móvil:* Las fotos del equipo se apilan en una sola columna vertical para evitar desbordamientos horizontales.



6. Funcionalidad de la página web.

Tras la definición del diseño, es importante desglosar cómo se comporta la página web ante las distintas acciones.

Hemos definido en el apartado anterior las páginas a las que accedemos mediante las distintas opciones del menú, en este apartado, implementamos la funcionalidad en laravel para permitir esas interacciones y que, al hacer click sobre el catálogo, nos lleve a dicha página.

Laravel utiliza la arquitectura de modelo-vista-controlador como ya hemos mencionado anteriormente, vamos a ilustrar con un ejemplo cómo se realizan estas acciones.

1. **El usuario realiza una petición.** Por ejemplo, el usuario se encuentra en la página de inicio y quiere acceder al catálogo de animales. Hace click en “Explorar repositorios” en el header, que contiene un enlace <a> hacia la página /catalogo.
2. **Rutas.** La petición llega a los archivos de rutas, que se encargan de vincular las rutas con los controladores necesarios para cargar la página. En este caso sería el controlador “AnimalController.php”
3. **Controlador.** El controlador ahora sabe que tiene que mostrar el catálogo, pero no tiene los datos, por lo que los pide al Modelo, en este caso “Animal.php”. El modelo se comunica con la base de datos y le da los datos que necesita al controlador. Tras esto, el controlador le pasa los datos a la vista html.
4. **Vista.** La página html recibe estos datos. Esto lo hace mediante la variable que necesita de la base de datos, y el código proporcionado por blade como

```
<div class="grid-animales">
  @foreach($animales as $animal)
    <div class="card-animal card-{{ strtolower($animal->species) }}">
      <div class="card-animal-inner">
        <div class="repo-image-wrapper">
          name }}">
          @auth
            @if(Auth::user()->role === 'user')
              <form action="{{ route('favoritos.toggle', $animal->id) }}" method="POST">
                @csrf
                <button type="submit" class="btn-star-fav {{ Auth::user()->favoritos($animal->id) ? 'active' : '' }}">
                  </form>
              @endif
            @endauth
          </div>
          <div class="card-info">
            <h3>{{ $animal->name }}</h3>
            <div class="etiquetas">
              <span class="etiqueta">{{ strtolower($animal->species) == 'perro' ? 'Perro' : 'Gato' }}</span>
              <span class="etiqueta">
                @if($animal->status == 'adopted')
                  Adoptado
                @elseif($animal->adoptionRequests->count() > 0 || $animal->status == 'pending')
                  Pendiente
                @else
                  Disponible
                @endif
              </span>
              <span class="etiqueta">🏠 {{ \Carbon\Carbon::parse($animal->birth_date)->format('d/m/Y')}}</span>
            </div>
          </div>
        </div>
      </div>
    </div>
  </foreach>
</div>
```

`{{}}`, `@foreach` o `@if`. De esta forma se pinta la información recibida. En esta imagen se incluye la parte “catalogo.blade.php” donde mediante un `@foreach` se reciben los animales y se van pintando en el catálogo empleando las directivas de blade. Luego, la página se embellece mediante el uso de css.

5. **Respuesta final.** Finalmente, la página html responde mostrando la información como la vemos en pantalla.

De esta forma gestionamos la funcionalidad de la página. Hay distintos controladores para distintas acciones y páginas pero de forma esencial todos funcionan de la misma forma.

7. Diseño de inicio de sesión y registro.

En los requisitos tenemos que definir unos roles muy concretos como son usuarios no identificados, usuarios y voluntarios, que serán los administradores en este contexto.

7.1 Registro.

El registro consta de un formulario creado donde el controlador “AuthController” comprobará si los datos rellenados son correctos:

```
class AuthController extends Controller
{
    public function register(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'username' => 'required|string|max:255|unique:users',
            'email' => 'required|string|email|max:255|unique:users',
            'role' => 'required|in:user,admin,voluntario',
            'password' => 'required|string|min:8|confirmed',
        ], [
            'name.required' => 'El nombre es obligatorio.',
            'username.required' => 'El nombre de usuario es obligatorio.',
            'username.unique' => 'Este nombre de usuario ya está en uso.',
            'email.required' => 'El correo electrónico es obligatorio.',
            'email.email' => 'El formato del correo no es válido.',
            'email.unique' => 'Este correo ya está registrado.',
            'password.required' => 'La contraseña es obligatoria.',
            'password.min' => 'La contraseña debe tener al menos 8 caracteres.',
            'password.confirmed' => 'Las contraseñas no coinciden.',
        ]);

        $user = User::create([
            'name' => $request->name,
            'username' => $request->username,
            'email' => $request->email,
            'role' => $request->role,
            'password' => Hash::make($request->password),
        ]);

        Auth::login($user);

        return redirect()->intended('/')->with('success', '¡Cuenta creada con éxito! Has iniciado sesión.');
```

En primer lugar, se establece el tipo y valores de cada uno de los campos, por ejemplo, que todos los campos son obligatorios, el tipo de dato que alojan, si son únicos y el máximo o mínimo de caracteres requeridos para dicho campo. Se verifica que dichos campos sean correctos y en caso contrario se lanza un aviso o mensaje de error, todo esto lo vemos definido en el apartado de validación de la petición.

En caso de que todo sea correcto, se lanza un mensaje create a la base de datos con los datos que se han introducido en el registro.

La contraseña se cifra con un hash para que no se almacene en texto plano en la base de datos lo cual sería peligroso si alguien consiguiera acceder a ella de forma externa.

Inmediatamente tras el registro, se aplica una orden login que inicia sesión al usuario automáticamente, impidiendo que tenga que introducir manualmente los datos.

7.2 Inicio de sesión.

El inicio de sesión funciona de forma similar al registro, solo que sin guardar nada en la base de datos.

En primer lugar el usuario introduce su username y contraseña, no valen otros datos. Se comprueba que se hayan introducido.

Se produce un intento de inicio de sesión, donde laravel busca en la base de datos si el usuario con el username indicado existe, en caso afirmativo compara las contraseñas. Si las credenciales coinciden, se inicia sesión de forma correcta y se le redirige a la página que se intentaba acceder.

Si no, se muestra en pantalla un fallo de inicio de sesión.

```
public function login(Request $request)
{
    $credentials = $request->validate([
        'username' => 'required|string',
        'password' => 'required|string',
    ], [
        'username.required' => 'El nombre de usuario es obligatorio.',
        'password.required' => 'La contraseña es obligatoria.',
    ]);

    if (Auth::attempt($credentials)) {
        $request->session()->regenerate();
        return redirect()->intended('/')->with('success', 'Has iniciado sesión correcta');
    }

    return back()->withErrors([
        'login' => 'Las credenciales proporcionadas no son correctas.',
    ]);
}
```

7.3 Cierre de sesión.

El cierre de sesión se activa pulsando el botón asignado de cierre de sesión en el header.

Al cerrar sesión, el usuario dejará de tener acceso a su vista personalizada propia, así como se invalidará dicha sesión para evitar que esta pueda ser usada nuevamente para iniciar sesión sin necesidad de usar las credenciales, lo cual puede ser problemático a nivel de seguridad y se regenera el token CSRF.

```
public function logout(Request $request)
{
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect('/')->with('success', 'Has cerrado sesión.');
```

7.4 Tipos de cuenta.

En el sistema existen 3 tipos de cuenta, **no identificado**, **usuario/adpotante** y **voluntario/admin**. Como hemos explicado anteriormente, cada rol tiene sus propias características que los hacen distintos del resto:

- **no identificado:** No dispone de menú lateral, tiene acceso limitado a las páginas y no puede interactuar con ellas más allá de la “lectura” de las mismas.
- **usuario/adpotante:** Se trata del usuario normal de la página web. Ya posee su cuenta y puede realizar acciones como enviar solicitudes para adoptar, ver el estado de las mismas, guardar como favoritos a los animales que se quieran y editar el perfil desde ajustes.
- **voluntario/admin:** El usuario de mayor poder en la página, no pueden realizar las acciones del usuario promedio, pero sí pueden crear una nueva oferta de adopción, modificar las ya existentes, leer los tickets de contacto proporcionados por los usuarios y aceptar o rechazar ofertas de adopción.

Todas estas funcionalidades se llevarán a cabo por medio del menú lateral, el cuál cambia de forma dinámica dependiendo del tipo de cuenta que se encuentre iniciada en la página.

Esta es la implementación del menú lateral, donde podemos ver cómo muestra unas opciones u otras dependiendo del rol que se encuentre activo en el momento, lo cual se puede hacer gracias a las directivas de blade.

```
<div class="contenido-aside">
    @guest
        <!-- Si no ha iniciado sesión -->
        <h3 class="mt-0">Tu Dashboard</h3>
        <p class="sidebar-p-small">
            Inicia sesión para darle <b>Star</b> a tus repositorios de animales favoritos y hacer seguimiento de tus <b>Pull Requests</b> (adopciones).
        </p>
    @endguest

    @auth
        <!-- Opciones exclusivas para Usuario Normal -->
        @if(Auth::user()->role === 'user')
            <h3 class="mt-0">Tus Repositorios</h3>
            <ul>
                <li><a href="/mis-favoritos"> Repositorios Starred</a></li>
                <li><a href="/mis-solicitudes"> Mis Pull Requests</a></li>
                <li><a href="/perfil"> Settings</a></li>
            </ul>
        @endif

        <!-- Opciones exclusivas de Voluntarios/Administradores -->
        @if(Auth::user()->role === 'voluntario' || Auth::user()->role === 'admin')
            <h3 class="mt-0">Organización (Admin)</h3>
            <ul>
                <li><a href="/animales/gestion" class="enlace-admin"><span class="plus-icon-admin"></span> Nuevo Repo</a></li>
                <li><a href="/solicitudes-pendientes" class="enlace-admin"> Merge Requests</a></li>
                <li><a href="/tickets" class="enlace-admin"> Tickets</a></li>
                <li><a href="/perfil"> Settings</a></li>
            </ul>
        @endif
    @endauth
</div>
```

8. Enlaces de interés.

ENLACE AL REPOSITORIO DE GITHUB:

<https://github.com/AngelMorExpo04/TW-PROTECTORA.git>

ENLACE AL WEB EN UN HOST:

<https://guauhub.alwaysdata.net/>